1. Асимптотика: определения и переходы

1 — О-символика 2 — сортировки и контейнеры 3 — циклы и корректность 4 — рекурсии

Что именно означает оценка*

$f, g > 0$ при больших n. Константы не зависят от n; все неравенства ниже верны при $n \ge n_0$.

$$\begin{aligned} f = O(g) &\iff \exists C > 0 :\ f \le Cg, \\ f = \Omega(g) &\iff \exists c > 0 :\ f \ge cg, \\ f = \Theta(g) &\iff \exists c, C > 0 :\ cg \le f \le Cg. \end{aligned}$$

В каждой строке также существует порог n_0. O — **верхняя**, Ω — **нижняя**, Θ — **точная по порядку** граница.

Для $g > 0$:

$$f = o(g) \iff f/g \to 0, \qquad f = \omega(g) \iff f/g \to \infty.$$

В o: для *каждого* $\varepsilon > 0$ в итоге $f \le \varepsilon g$. В ω: для *каждого* $C > 0$ в итоге $f \ge Cg$.

Равносильные переходы*

$$\begin{aligned} f = O(g) &\iff g = \Omega(f), \\ f = \Omega(g) &\iff g = O(f), \\ f = \Theta(g) &\iff g = \Theta(f), \\ f = \Theta(g) &\iff \begin{cases} f = O(g), \\ f = \Omega(g), \end{cases} \\ &\iff \begin{cases} f = O(g), \\ g = O(f), \end{cases} \\ f = o(g) &\iff g = \omega(f). \end{aligned}$$

Для классов функций:

$$O(f) = O(g) \iff f = \Theta(g) \iff \Omega(f) = \Omega(g).$$

Следствия: $\Theta(g) \subseteq O(g)$ и $\Theta(g) \subseteq \Omega(g)$. Все пять отношений транзитивны.

Не равносильно: $f = O(g) \not\Rightarrow g = O(f)$. Верхняя и нижняя границы не обязаны быть точными.

Дроби и степени

При $f, g > 0$ и постоянном $r > 0$:

$$\begin{aligned} f = O(g) &\iff f/g = O(1) \\ &\iff 1/g = O(1/f) \\ &\iff f^r = O(g^r). \end{aligned}$$

При обращении дробей O и Ω меняются местами: $f = O(g) \iff 1/f = \Omega(1/g)$. Для Θ степень и обращение также сохраняют эквивалентность.

Как доказать Θ по определению*

Найти $c_1, c_2 > 0, n_0$ и показать

$$c_1 g(n) \le T(n) \le c_2 g(n) \quad (n \ge n_0).$$

Одного $T(n) \le c_2 g(n)$ недостаточно: это только $O(g)$.

Алгебра О-символов*

Для неотрицательных функций и постоянного $c > 0$:

$$\begin{aligned} O(f) + O(g) &= O(f + g) = O(\max\{f, g\}), \\ O(f)\,O(g) &= O(fg), \\ c\,O(f) &= O(f). \end{aligned}$$

Те же правила верны для Θ. В конечной сумме оставляем максимальный порядок; при произведении перемножаем порядки.

$$f = O(g) \iff f + g = \Theta(g).$$

$f = O(g)$ и $u = O(v) \Rightarrow fu = O(gv)$. Сложение и умножение сохраняют также нижние оценки для неотрицательных функций.

Вычитание: ведущие члены могут сократиться. Из $f, g = \Theta(n)$ нельзя заключить $f - g = \Theta(n)$.

Сравнение через отношение

Если существует $L = \lim f(n)/g(n)$, $g > 0$:

$L = 0$	$f = o(g),\ f = O(g),\ f \notin \Omega(g)$
$0 < L < \infty$	$f = \Theta(g)$
$L = \infty$	$f = \omega(g),\ f = \Omega(g),\ f \notin O(g)$

Предел может не существовать, хотя Θ верна. Полный критерий для $f, g > 0$:

$$f = \Theta(g) \iff 0 < \liminf \frac{f}{g} \le \limsup \frac{f}{g} < \infty.$$

Логарифмы и шкала роста*

$b > 1$, a, r, k, ε — постоянные:

$$\log_b n = \frac{\ln n}{\ln b}, \qquad \log(n^r) = r \log n.$$

$$\log^r n = (\log n)^r \ne \log(n^r).$$

$$(\log n)^k = o(n^\varepsilon) \quad (k, \varepsilon > 0).$$

$$1 \prec \log n \prec n \prec n \log n \prec n^{3/2} \prec n^2,$$

$$n^k \prec n^{\log n} \prec a^n \prec n! \quad (k > 0,\ a > 1).$$

Здесь $\prec$ означает строго меньший порядок. Полином с положительным старшим коэффициентом имеет порядок старшей степени.

Ловушки в тестах*

$O \ne \Theta$. Для $T = \Theta(n \log n)$ верны $O(n^2)$ и $\Omega(n)$, но неверны $\Theta(n)$ и $\Omega(n^2)$.

$\Omega(1) \ne O(1)$. $\Omega(1)$ может быть и n, и 2^n. Поэтому $O(n) + \Omega(1)$ не даёт верхней оценки $O(n)$.

Случай и граница — разные вещи. O не означает «худший случай», а Ω — «лучший». Оценку применяют к уже выбранной функции времени.

$f = O(g)$ — не обычное равенство. Это принадлежность классу; переставлять стороны как в числовом равенстве нельзя.

2. Сортировки и линейные структуры

Время — для стандартных реализаций; сравнение ключей за $\Theta(1)$. Дополнительная память не включает входной массив.

Сводная таблица сортировок

Алгоритм	Лучшее	Среднее	Худшее	Доп. память	Устойчив.
Insertion / вставки*	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	да
Selection / выбором*	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	нет
Bubble без флага*	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	да
Bubble с флагом*	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$	$O(1)$	да
Merge / слиянием*	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	да
Quick / быстрая*	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$O(\log n)$ / $O(n)$	нет
Shell: шаги $n/2, n/4, \dots$	$\Theta(n \log n)$	см. ниже	$\Theta(n^2)$	$O(1)$	нет
Heap / пирамидальная	$\Theta(n \log n)^\dagger$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	нет
Counting / подсчётом	$\Theta(n + K)$	$\Theta(n + K)$	$\Theta(n + K)$	$O(n + K)$	да
LSD Radix / поразрядная	$\Theta(D(n + B))$	$\Theta(D(n + B))$	$\Theta(D(n + B))$	$O(n + B)$	да
Bucket / блочная	$\Theta(n)$	$\Theta(n)$	$\Theta(n^2)$	$O(n)$	условно

Устойчивость: равные ключи сохраняют исходный взаимный порядок. Среднее для сравнительных сортировок — по случайным перестановкам; для quick sort также подходит случайный выбор опорного при различных ключах.
†Heap: различные ключи; с равными лучший случай зависит от реализации.

Состояние массива после прохода*

Insertion. После k итераций (первая обрабатывает второй элемент) первые $k + 1$ исходных элементов отсортированы; остальной суффикс не обработан. Число сдвигов равно числу инверсий I; $T = \Theta(n + I)$, инверсия: $i < j$, $a_i > a_j$.
Bubble. При проходе слева направо максимум уходит вправо. После k проходов k последних элементов на своих местах. Без флага раннего выхода лучший случай тоже квадратичный.
Merge. Две отсортированные части сливаются за $\Theta(n)$; при равенстве брать из левой части для устойчивости.

Quick sort: опорный и разбиение*

Если опорный поставлен на окончательное место p, то

$$\max_{i<p} a_i \le a_p \le \min_{i>p} a_i.$$

Оба условия обязательны; части внутри могут быть не отсортированы. В схеме Hoare возвращается граница, опорный не обязан стоять на окончательном месте.
Баланс частей $\Rightarrow \Theta(n \log n)$; размеры $0, n-1 \Rightarrow \Theta(n^2)$. Стек: обычно $O(\log n)$, худший $O(n)$. Рекурсия только в меньшую часть гарантирует $O(\log n)$ стека.
Quick часто быстрее Merge из-за накладных расходов и отсутствия отдельного массива, а не гарантированно меньшего числа сравнений.

Shell и цена сравнения

Shell — вставки с убывающими шагами. Средняя оценка зависит от последовательности. Для шагов Хиббарда / Кнута верхняя граница $O(n^{3/2})$, для последовательности Седжвика из семинара — $O(n^{4/3})$.
Если сравнение стоит $O(L)$, Merge требует $O(nL \log n)$. Для строк L — максимальная длина. Сравнительным сортировкам в худшем случае нужно $\Omega(n \log n)$ сравнений.

Массив, список, стек*

Худший случай; известный узел не нужно искать. Для массива: порядок сохраняется, без перевыделения.

	Массив	1-связ.	2-связ.
Доступ по индексу	$O(1)$	$O(n)$	$O(n)$
Поиск значения	$O(n)$	$O(n)$	$O(n)$
Вставка в середину	$O(n)$	$O(1)^a$	$O(1)^a$
Удалить узел	$O(n)$	$O(1)^b$	$O(1)^a$
Удалить хвост	$O(1)$	$O(n)$	$O(1)^c$

^{a}Известно место / узел. ^{b}Известен также предшественник. ^{c}Есть указатель на хвост.
Вставка в массив со сдвигом: $\Theta(n - i + 1)$. Добавление в конец при удвоении ёмкости: $O(1)$ амортизированно, отдельный вызов $O(n)$.
В односвязном списке k-й от головы: $\Theta(k)$, для фиксированного k — $\Theta(1)$; k-й от конца — $\Theta(n)$.
Стек — LIFO; очередь — FIFO. Операции на связной реализации $O(1)$ (у очереди есть голова и хвост).

Условия для линейных сортировок

Counting: считаем частоты целых чисел. K — число значений в диапазоне $0, \dots, K-1$. Время $\Theta(n + K)$; линейно при $K = O(n)$. В таблице — версия с выходным массивом.
LSD Radix: сортируем по цифрам от младшей к старшей. D — число разрядов, B — основание ($B = 10$ для десятичных чисел). Линейно при постоянных D, B. Проходы — устойчивый подсчёт.
Bucket: разложить по n равным интервалам, в каждом отсортировать вставками и соединить. Независимые равномерные данные дают $\Theta(n)$ в среднем; худшее $\Theta(n^2)$. Для устойчивости не менять порядок равных.

Короткие теоретические факты

АТД — операции и их смысл; структура данных — реализация.
Двусвязный список: больше памяти, обход в обе стороны. Рекурсивный вывод после обхода следующего узла: обратный порядок; время $\Theta(n)$, стек $\Theta(n)$.
Линейный порядок: рефлексивность, антисимметричность, транзитивность; любая пара сравнима.

3. Циклы, корректность и выбор алгоритма

Сначала считаем реальные затраты; затем упрощаем сумму. Число уровней или итераций само по себе не равно времени.

От изменения счётчика к числу шагов*

Постоянные: целое $b > 1$ и $d > 0$.

Значения счётчика	Число шагов
$a, a+d, \ldots < M$	$\lceil (M-a)/d \rceil$
$1, b, b^2, \ldots < M$	$\lceil \log_b M \rceil$
$n, \lfloor n/b \rfloor, \ldots \geq 1$	$\lfloor \log_b n \rfloor + 1$

Первая строка: $M > a$; вторая: $M > 1$.
Последовательные части: $T = T_1 + T_2$.
Вложенные проходы: $N = \sum_i N_i$. Умножать число внешних шагов на одну внутреннюю стоимость можно, если она одинакова на всех шагах.

Суммы, которые дают точный порядок*

$$\sum_{i=1}^{m} i = \frac{m(m+1)}{2},$$

$$\sum_{t=0}^{h-1} q^t = \frac{q^h - 1}{q - 1} \quad (q \neq 1),$$

$$\sum_{t=0}^{\infty} q^t = \frac{1}{1-q} \quad (|q| < 1).$$

При $q > 1$ порядок задаёт последний член; при $0 < q < 1$ — первый.

Зависимая внутренняя работа*

n **внешних шагов; внутренние длины** $1, 2, 4, \ldots$. При $h = \lceil \log_2 n \rceil$:

$$N = n \sum_{t=0}^{h-1} 2^t = n(2^h - 1).$$

$$n \leq 2^h < 2n \;\Rightarrow\; \tfrac{1}{2}n^2 \leq N < 2n^2 \quad (n \geq 2).$$

Значит, $\Theta(n^2)$; $c_1 = 1/2, c_2 = 2, n_0 = 2$.
Растущий счётчик; цена шага $b^t - 1$**.** $h = \lceil \log_b M \rceil$, $b > 1$ постоянно:

$$N = \sum_{t=0}^{h-1} (b^t - 1) = \frac{b^h - 1}{b - 1} - h = \Theta(M).$$

Если $M = cn^\alpha$, $c, \alpha > 0$ постоянны, то $T = \Theta(n^\alpha)$.

Каскад с делением аргумента на b

Для $\alpha > 0$ и постоянного $b > 1$:

$$\sum_{t=0}^{\lfloor \log_b n \rfloor} (n/b^t)^\alpha = \Theta(n^\alpha).$$

Цена одного шага	Весь каскад
$\Theta(1)$	$\Theta(\log n)$
$\Theta(x^\alpha),\ \alpha > 0$	$\Theta(n^\alpha)$
$\Theta(\log x)$	$\Theta(\log^2 n)$

Цена $x, x^2, \sqrt{x}$ даёт порядок $n, n^2, \sqrt{n}$. При цене $\log x$ суммируется арифметическая прогрессия.

Время и число выбранных операций*

$$T(n) = \sum_j c_j N_j(n).$$

В RAM машинная операция стоит $\Theta(1)$; для строк и длинных чисел проверять цену отдельно.
Лучшее / худшее: минимум / максимум по входам размера n. **Среднее:** ожидание по распределению. **Амортизированное:** стоимость последовательности / число операций; случайность не нужна.
Если при просмотре n элементов запись выполняется лишь для r блоков одинаковых соседних значений, число записей равно r, но полное время $\Theta(n)$, даже при $r = 1$.

Инвариант: что проверять*

Инвариант P**:** истинен перед первой итерацией и перед каждой следующей проверкой условия.

1. **Инициализация:** предусловие $\Rightarrow P$.
2. **Сохранение:** P и условие цикла $\Rightarrow P$ после тела.
3. **Выход:** P и отрицание условия $\Rightarrow$ нужный результат.

Завершение доказывается **отдельно**: неотрицательная целая величина строго убывает.
Примеры: перед шагом i сумма $S = \sum_{j=1}^{i-1} j$; префикс до i отсортирован; в просмотренном префиксе цели нет.
Инвариантов может быть несколько. $2 = 2$ — допустимый, но бесполезный инвариант. Внутри тела между присваиваниями P может нарушаться.

Практический выбор алгоритма*

$$T_A = an \log_b n, \quad T_B = dn, \quad a, d > 0:$$

$$T_A < T_B \iff \log_b n < d/a \iff n < b^{d/a}.$$

Сначала найти порог, затем сравнить с **допустимым диапазоном**.
Если принята модель $T = cf(n)$ и известно $T(n_0) = t_0$, то

$$c = \frac{t_0}{f(n_0)}, \qquad T(n) = t_0 \frac{f(n)}{f(n_0)}.$$

Из одного $O(f(n))$ такую точную модель получить нельзя.

Независимые повторные запуски*

Успех одного запуска — $p \in (0, 1)$:

$$P(\text{хотя бы один успех}) = 1 - (1-p)^k.$$

Для требуемой вероятности $q \in (0, 1)$ положим $R = \ln(1-q)/\ln(1-p)$:

$$P \geq q:\ k_{\min} = \lceil R \rceil; \qquad P > q:\ k_{\min} = \lfloor R \rfloor + 1.$$

Строгое «превысила» отличается от «не меньше». При делении на $\ln(1-p) < 0$ знак меняется. Успехи должны быть независимы.

4. Рекурсии: выбрать метод и получить оценку

Сначала записать все реально выполняемые вызовы, нерекурсивную работу и базу. Глубина дерева не равна полному времени.

Дерево равных подзадач*

$T(n) = aT(n/b) + f(n)$; целое $a \geq 1$ и $b > 1$ постоянны, база $\Theta(1)$. Для $n = b^h$:

Высота	$h = \log_b n$
Узлов на уровне i	a^i
Размер задачи	n/b^i
Работа уровня	$a^i f(n/b^i)$
Число листьев	$a^h = n^{\log_b a}$

$$T(n) = \sum_{i=0}^{h-1} a^i f(n/b^i) + \Theta(a^h).$$

Если размер уменьшается на d, а не в b раз, высота $\Theta(n/d)$, а не $\Theta(\log n)$.

Мастер-теорема*

$T(n) = aT(n/b) + f(n)$, $p = \log_b a$, $f \geq 0$. **Сравнивать $f(n)$ с n^p.**

1. Доминируют листья. Для некоторого постоянного $\varepsilon > 0$:

$$f = O(n^{p-\varepsilon}) \quad \Rightarrow \quad T = \Theta(n^p).$$

2. Равные порядки по уровням. Для постоянного $k \geq 0$:

$$f = \Theta(n^p \log^k n) \Rightarrow T = \Theta(n^p \log^{k+1} n).$$

3. Доминирует верх дерева. Для некоторых $\varepsilon > 0$ и $q < 1$:

$$\left.\begin{array}{l} f = \Omega(n^{p+\varepsilon}), \\ af(n/b) \leq qf(n) \end{array}\right\} \Rightarrow T = \Theta(f(n)).$$

В третьем случае обязательно **условие регулярности**. В первом и третьем нужен разрыв на степень n^ε, а не просто на логарифм.

Не применяется напрямую к $T(n-d)$, разным долям подзадач или отрицательной стоимости f.

Подстановка = доказательство индукцией*

Гипотеза: $T(n) \leq Cg(n)$ (или $Cg(n) + A$).

1. Выбрать константы, закрыть все базовые случаи.
2. Подставить гипотезу для меньших аргументов.
3. Получить то же неравенство для n.

Для Ω доказывается обратное неравенство; для Θ — оба. Если шаг не замыкается, иногда нужна усиленная гипотеза, например $Cn - D$. Индукция не гарантирует, что выбрана самая точная граница.

«Разделяй и властвуй»*

Разделить → решить части → объединить. Части должны вести к базе, но не обязаны быть равными или уменьшаться в целое число раз. Независимые части можно распараллелить. Общей гарантии эффективности для любой задачи нет.

Постоянный шаг: развёртывание и база*

$T(n) = aT(n-d)+c$, $n = r+kd$, r — базовый аргумент:

$$T(r + kd) = a^k T(r) + c \sum_{j=0}^{k-1} a^j.$$

При $a \neq 1$, положив $L = c/(1-a)$:

$$\boxed{T(r + kd) = L + a^k(T(r) - L).}$$

При $a = 1$: $T(r + kd) = T(r) + kc$.

Сначала подставить базу! Если $T(r) = L$, экспонента сокращается: $T(n) \equiv L$. При $d > 1$ нужна база для каждого остатка по d. Отрицательная добавка требует проверки неотрицательности формального решения.

Частые рекурсии

База положительна и имеет порядок $\Theta(1)$.

Соотношение	$T(n)$
$T(n/2) + \Theta(1)$	$\Theta(\log n)$
$T(n/2) + \Theta(\log n)$	$\Theta(\log^2 n)$
$T(n/2) + \Theta(n)$	$\Theta(n)$
$2T(n/2) + \Theta(n)$	$\Theta(n \log n)$
$4T(n/2) + \Theta(n^2)$	$\Theta(n^2 \log n)$
$7T(n/2) + \Theta(n^2)$	$\Theta(n^{\log_2 7})$
$T(n-1) + \Theta(n)$	$\Theta(n^2)$
$2T(n-1) + \Theta(n)$	$\Theta(2^n)$

При одном вызове с делением на 2 и работе $\log n$:

$$m + (m-1) + \cdots + 1 = \Theta(m^2), \qquad m = \log_2 n.$$

Для Штрассена $\log_2 7 \approx 2.807 < 3$. Меньшая степень не гарантирует выигрыша на маленьких матрицах.

Разные доли: суммировать размеры*

$$T(n) = T(n/2) + T(n/4) + T(n/8) + cn, \quad c > 0.$$

Сумма долей $\rho = 1/2 + 1/4 + 1/8 = 7/8 < 1$. Работа активных узлов уровня i не больше $cn\rho^i$:

$$\sum_{i \geq 0} cn\rho^i = \frac{cn}{1-\rho} = 8cn.$$

Листья дают $O(n)$; корень уже даёт $\Omega(n)$.

$$\boxed{T(n) = \Theta(n).}$$

Индукция: $T(n) \leq A\rho n + cn \leq An$ при $A \geq c/(1-\rho)$ и выполненной базе.

Короткие ветви делят на 8, длинные — на 2; высоты порядка $\log_8 n$ и $\log_2 n$. На глубоких уровнях уже **не все 3^i узлов активны**.

Сравнительная рекурсия — только граница*

При неубывании T и согласованных базах:

$$3T(n/8) + cn \leq T(n) \leq 3T(n/2) + cn.$$

Сравнение даёт $\Omega(n)$ и $O(n^{\log_2 3})$. Верхняя оценка **не точна**: сумма по уровням улучшает её до $O(n)$. При разных долях подставлять одно произвольное b в мастер-теорему нельзя.